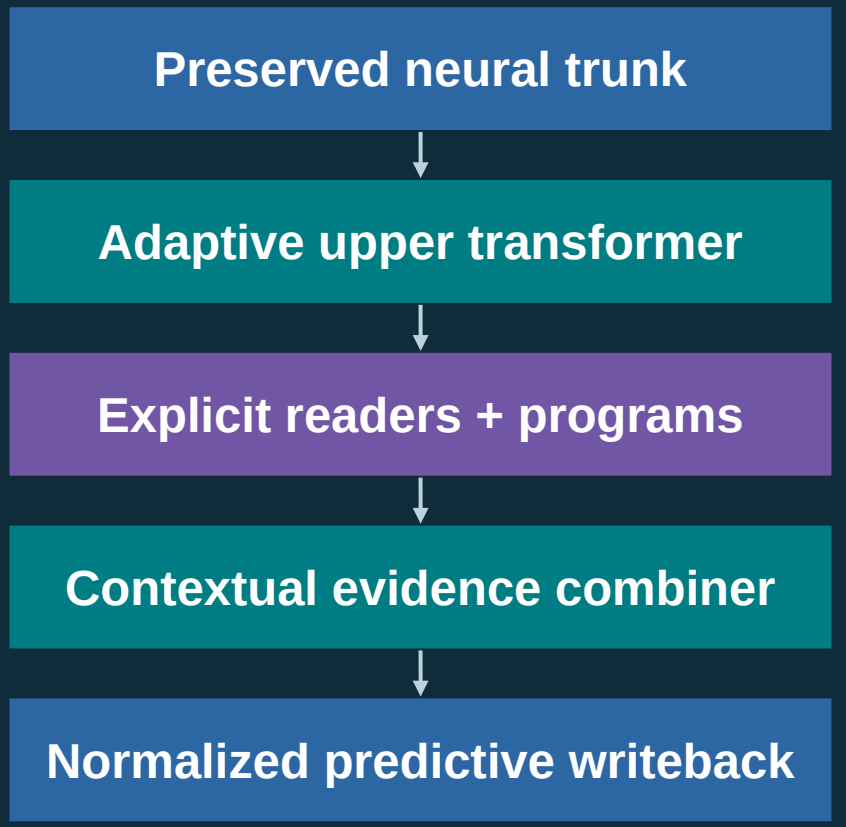
NESS

NEural-Symbolic Sandwich

Reusable programs inside an adaptive predictor.

Architecture / evidence / the next experiments



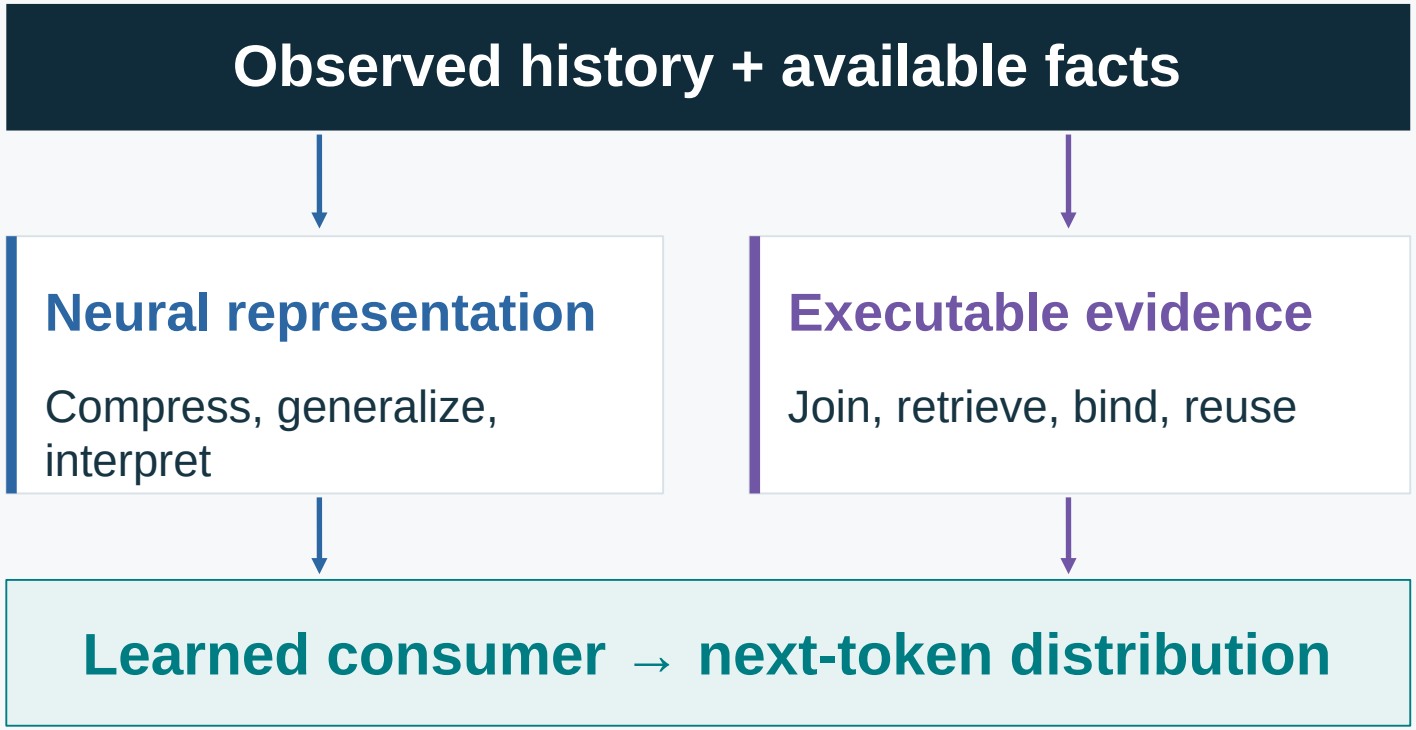
One prediction. Several kinds of computation.

Benjamin Goertzel · 20-minute technical briefing

September 2026

Expose the computation—not just more features

Explicit structure should make a useful computation accessible to the predictor.



Same facts. Different computational access.

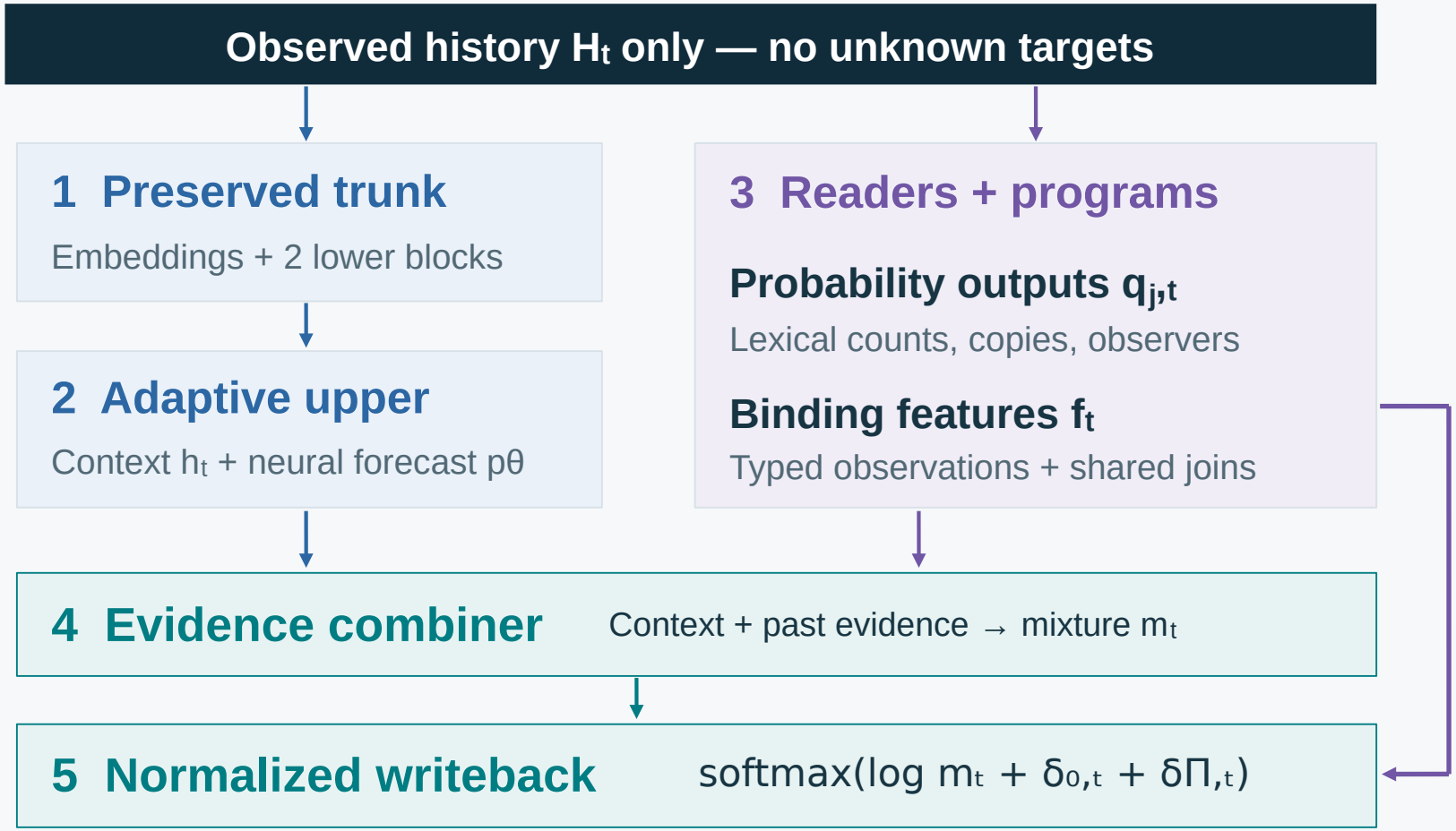
A finite consumer may benefit even when the program is deterministic from the history.

Composition-specific controls must receive the same observations—not a weaker information supply.

Sources: Paper 1 §§1–3; Paper 2 §9; Paper 4 §§1–2.

Five interfaces, not five transformer blocks

The current reference has parallel evidence paths, plus a direct binding-feature connection to the writeback head.



REFERENCE: v43–v45

Graph memory + soft composition provide a neural bridge over the same bindings.

Binding features bypass the mixture.

They change the final learned correction—not individual reader votes.

Sources: Paper 1 Fig. 1; Paper 2 §§3–5; implementation audit. Current reference: H3 v43–v45.

Two symbolic contracts; one normalized prediction

A grammar supplies executable computations; the consumer gives their outputs predictive meaning.

Probability-valued reader

Normalized next-token forecast
Learned counts, support and backoff

$q\pi(\cdot \mid H_t) \in$ probability simplex

Feature-valued binding program

Shared-witness test or structured fact
Prediction learned by the consumer

$f\pi,t = \text{execute}(\pi, B_t)$

$$m_t(y) = \sum_j w_{j,t}\, q_{j,t}(y) \qquad p_t = \text{softmax}(\log m_t + \delta_{o,t} + \delta_{\Pi,t})$$

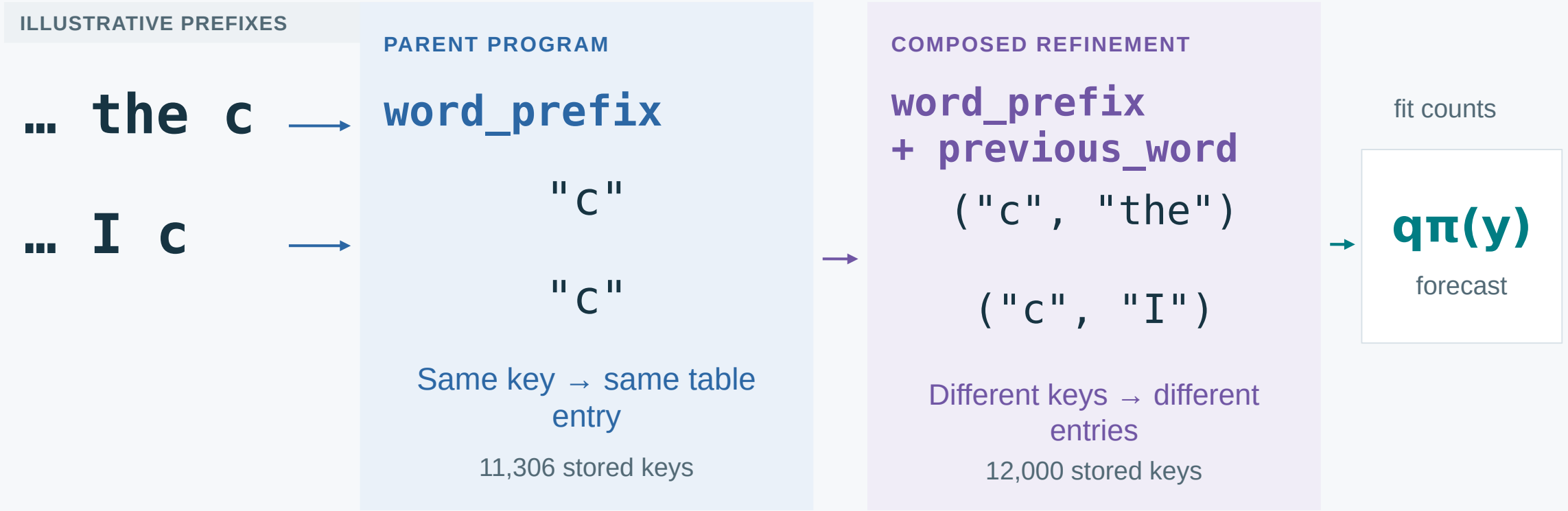
Mixture: choose among forecasts

Writeback: express a new calibrated combination

Sources: Paper 2 §§3–4. General framework; specific grammar is replaceable.

A learned reader can refine another reader

Original H3 discovered both observer forms; their output is a learned next-byte distribution.



A new conditional distribution—not just another weight. Sparse child evidence backs off to the parent.

Sources: original H3 reader report; Paper 2 §3.3; Paper 3 §3. Illustrative prefixes; actual discovered program forms.

Learn the observer and its predictive table separately

An executable observer defines a key; training outcomes teach the forecast attached to that key.

1 RUN THE OBSERVER

Training prefix → key

(word_prefix, previous_word)
→ ("c", "the")

2 COUNT OUTCOMES

After the next byte is observed

Increment n(key, byte).
No target enters the key.

3 SMOOTH + BACK OFF

Produce a full distribution

Sparse child → blend with parent
Unseen child → parent

$$q\pi(y) = [n(k,y) + \alpha\, q_parent(y)] / [n(k) + \alpha]$$

Separately: search which observations define the key

Original H3: 700,000 count-fit bytes · 98 tested programs → 2 retained readers.
Choose refinements using held-out predictive utility—not training counts alone.

Reader tables are learned statistics. v44–v45 freeze them; earlier experiments also tested refreshing their evidence.

Sources: Paper 2 §3.3; Paper 3 §3; original H3 reader report. Count fitting and structural selection are separate.

Separate state inference from parameter learning

NESS can support PC or BP. Do not confuse causal state settling with the weight-update rule.

CURRENT REFERENCE

Neural prior
u_t from context

→

Observed evidence
M: 8 past bytes

→

Settle reliability
8 logit updates

→

Predict
No target leakage

$$E_0(z) = \tfrac{1}{2}\|z - u\|^2 - \lambda \times \text{mean}_i \log(\sum_j w_j M_{ij})$$

v43–v45: BP through finite inference

Only reliability logits settle. Bindings and prior-network hidden units do not.

Earlier H3: nonlinear PC activity cap

Hidden activities + logits settle; centered nudges approximate parameter credit.

Sources: Paper 2 §4.1 and §8; Paper 3 §§3–4. Current and historical algorithms are distinct.

Three programs, three different computations

Actual selected v43 patterns. Repeated variables require identical entities—not independent matches.

ID 0 · DIRECT ACCESS

The actor has the right key

```
holds(A,K)
opens(K,C)
inside(O,C)
```

3 relations; same K and C

ID 398 · HELPFUL HOLDER

A helper has the right key

```
helps(H,A)
holds(H,K)
opens(K,C)
inside(O,C)
```

4 relations; same H, K and C

ID 681 · TRUSTED HELPER

That same helper is trusted

```
trusts(A,H)
helps(H,A)
holds(H,K)
opens(K,C)
inside(O,C)
```

5 relations; adds trusts(A,H)

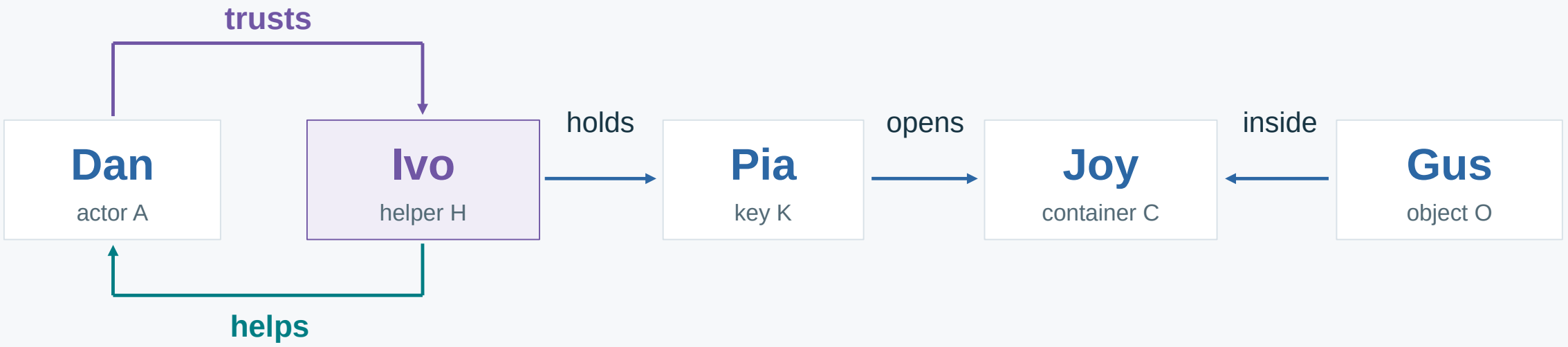
A = actor O = sought object H = helper K = key C = container

All listed relations must hold together. This is a semantic decomposition, not a claimed parent→child search history.

Source: v43 catalogue IDs 0 / 398 / 681 and search selections; Paper 2 §6. Decomposition is explanatory, not a search lineage.

A held-out world: Ivo completes the join

Dan seeks the object Gus. Routine mode. The five relevant edges below are from the archived test world.



Dan trusts	∩	Helps Dan	∩	Has a key for Gus	=
{Ivo}		**{Ivo}**		**{Kai, Ivo}**	**{Ivo}**

Program 681 returns 1: one identical helper satisfies trust, help, and the key–container–object chain.

Source: archived v43 test pair 2, row 4; program 681. Actual names and relations; only relevant edges are shown.

Same names. Same local text. Different binding.

Counterfactual pair 2: swap two trust endpoints; keep all other observed relations unchanged.

ORIGINAL WORLD

Dan trusts Ivo. **Eli trusts Eli.**

SWAPPED WORLD

Dan trusts Eli. **Eli trusts Ivo.**

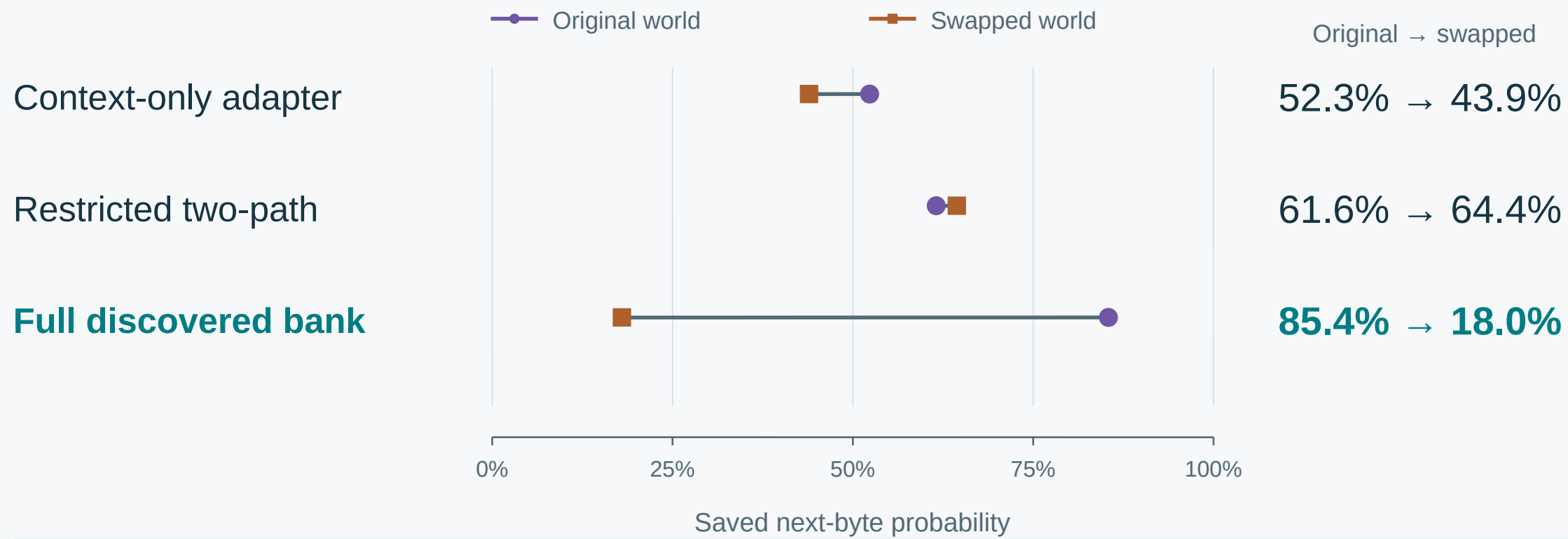
Executable test / endpoint set	Original world	Swapped world
Trusted by Dan	{Ivo}	{Eli}
Helps Dan	{Ivo}	{Ivo}
Owns a suitable key	{Kai, Ivo}	{Kai, Ivo}
ID 398: helpful holder	1	1
ID 681: trusted shared helper	**1**	**0**

Same cropped prefix: “… Qin. Dan seeks Gus. Routine. Then ”

Source: archived v43 pair 2, rows 4–5. Same local prefix and word counts; relation in/out degrees verified unchanged.

The shared binding changes the next prediction

P(next byte = “A”, beginning “Asks aid.”) · mean of three saved v43 models · the same held-out pair



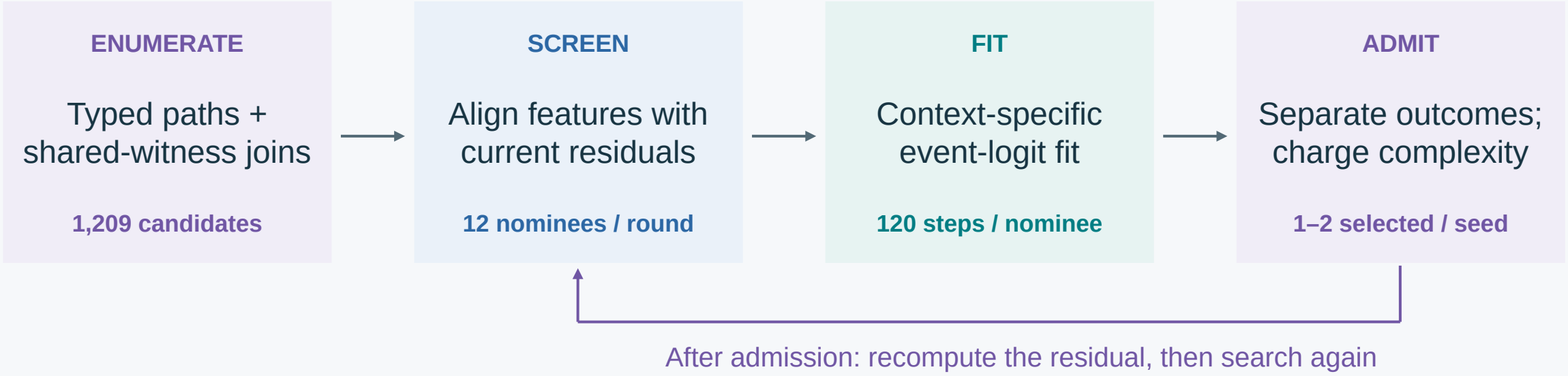
Evaluation-only generator reference: 88.4% → 17.2%. Not used as training labels.

One illustrative pair, selected using observations only. These are not full-continuation probabilities.

Source: saved v43 first-action forecasts, pair 2; mean of 3 seeds. One illustrative pair, not an aggregate result.

Search learns which binding computations to keep

A typed grammar supplies candidates. Prediction errors determine which computations become useful additions.



Learn syntax by search—not by backprop

Program 681 retained in all three full searches.

Learn consequences with a consumer

Final adapter + test score all 29 continuation bytes.

v43 uses an event-logit search surrogate. v44–v45 keep the discovered programs fixed while fitting neural weights.

Sources: Paper 2 §§6–7; v43 search43.py. Search fits a small surrogate; final training and scoring use all 29 bytes.

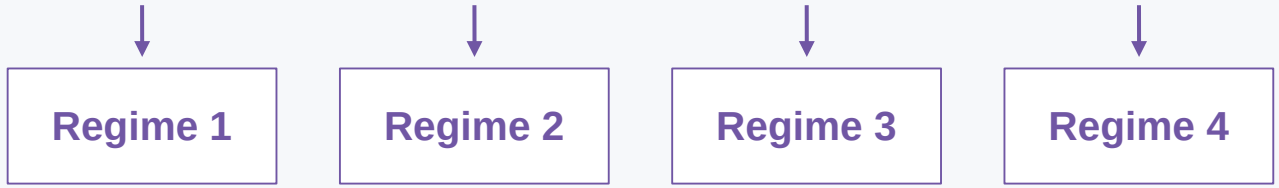
Combiner learning: probabilistic choice + predictive credit

A concrete earlier combiner: choose a weighting regime, score program evidence, then sum over the choices.

EARLIER H3 v16 · FOUR-STATE TEMPLATE

Context → learned gate g(c)

Probabilities over four latent weighting regimes



Each regime normalizes a signed, program-weighted correction of the baseline

$$p(y) = \Sigma_s \, g_s(c) \, q_s(y)$$

WHAT IS LEARNED?

Contextual gate + evidence weights

Reveal the next training byte. Minimize −log p(next byte) plus regularization.

Fixed logic; learned probabilities.
No correct-regime labels needed.

Current v43–v45: a learned reliability prior + finite settling + program-feature correction, not this four-state graph.

Sources: H3 v16 contextual-memory report; Paper 2 §4.2. Four-state example is historical, not the v43–v45 graph.

A deliberately small, controlled mechanism test

v44 tests a fixed discovered program bank under exact-initialization, matched-update controls.

Neural reference

Width 96; 4-head attention
2 frozen lower blocks
1 adaptive compressed upper
16 probability-reader slots

453,720

deployed parameter tensors

Task and measurement

20 entities; 9 relation types
35-byte local prefix
29-byte complete continuation
3 stochastic event decisions

All 29 bytes

not only selected decision positions

Controlled experiment

Frozen / joint core
None / full / two-path / corrupt
3 seeds × 8 branches
600 matched updates per branch

24 branches

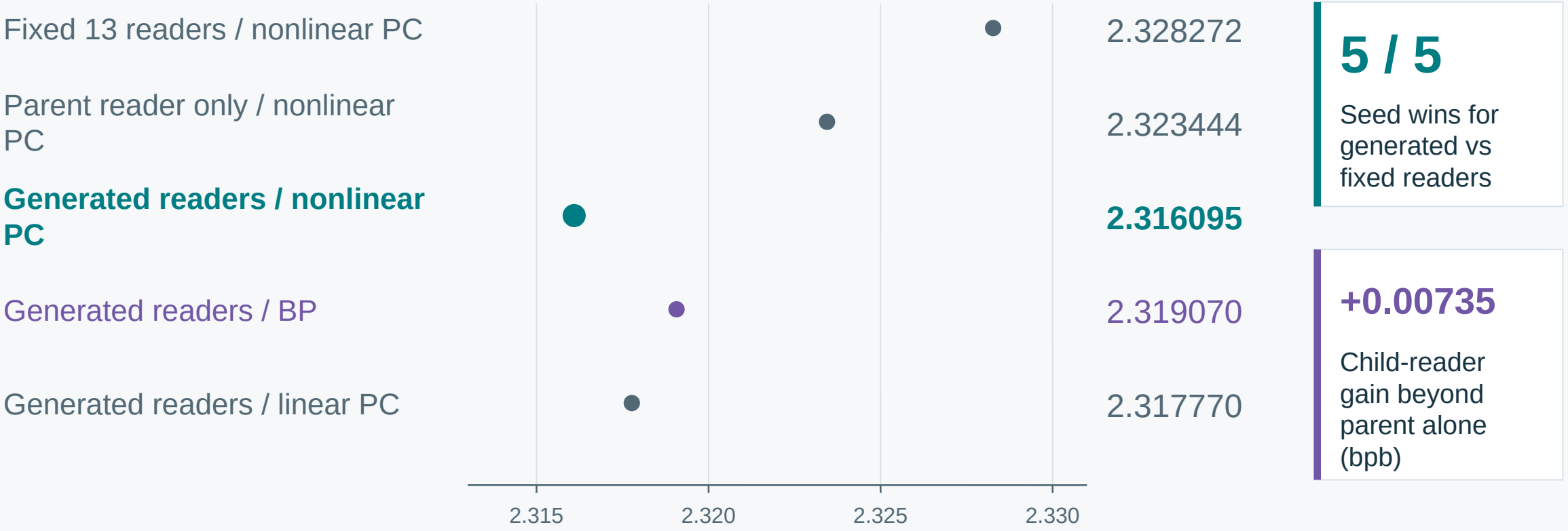
same initial predictions within seed

254,400 frozen + 199,320 update-eligible parameters. Same observations and training schedules; not equal FLOPs.

Sources: Paper 3 §2 and §11; implementation audit. Counts exclude fixed buffers and external memory.

Generated readers earned a predictive role

Original H3 · Tiny Shakespeare · five cap seeds · first held-out segment shown



5 / 5

Seed wins for generated vs fixed readers

+0.00735

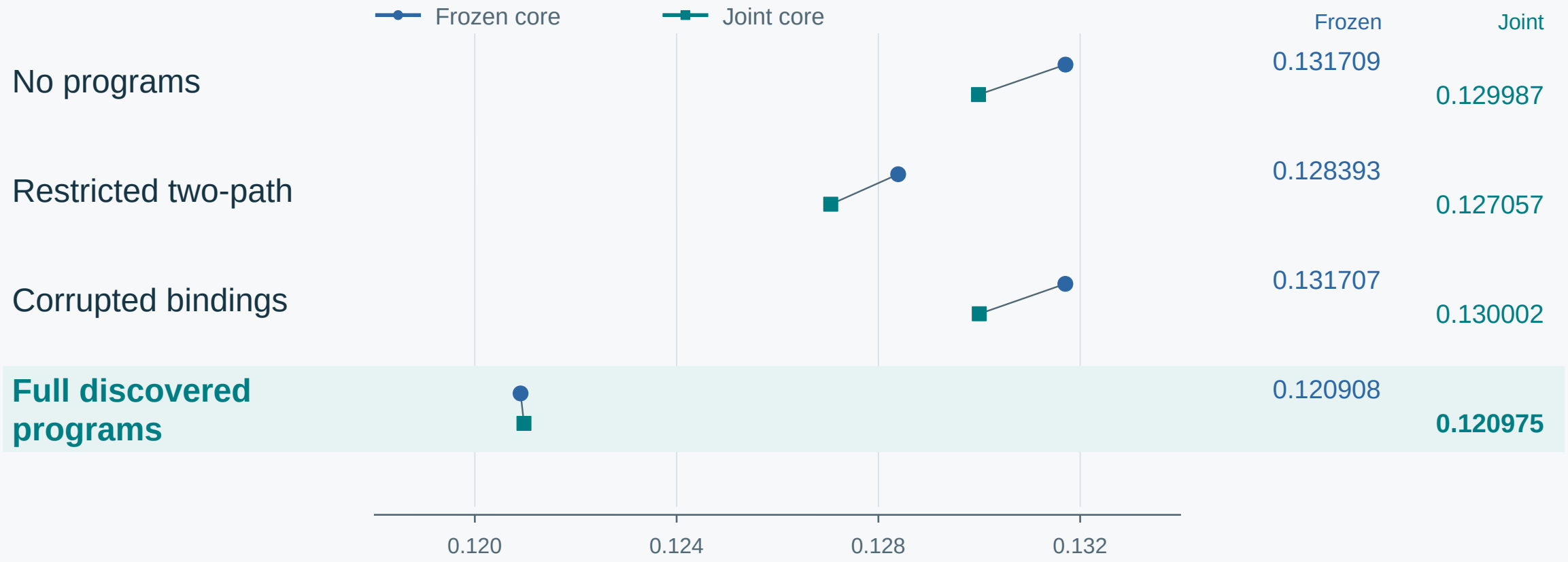
Child-reader gain beyond parent alone (bpb)

Reader gains hold on both segments. Nonlinear vs linear is mixed; PC/BP differences are small and setting-dependent.

Source: Paper 3 §3 / original H3 report. Historical report-derived values; common corpus and backbone.

Binding programs improve complete continuations

v44 · fixed endpoint · full structured-continuation loss · three-seed means

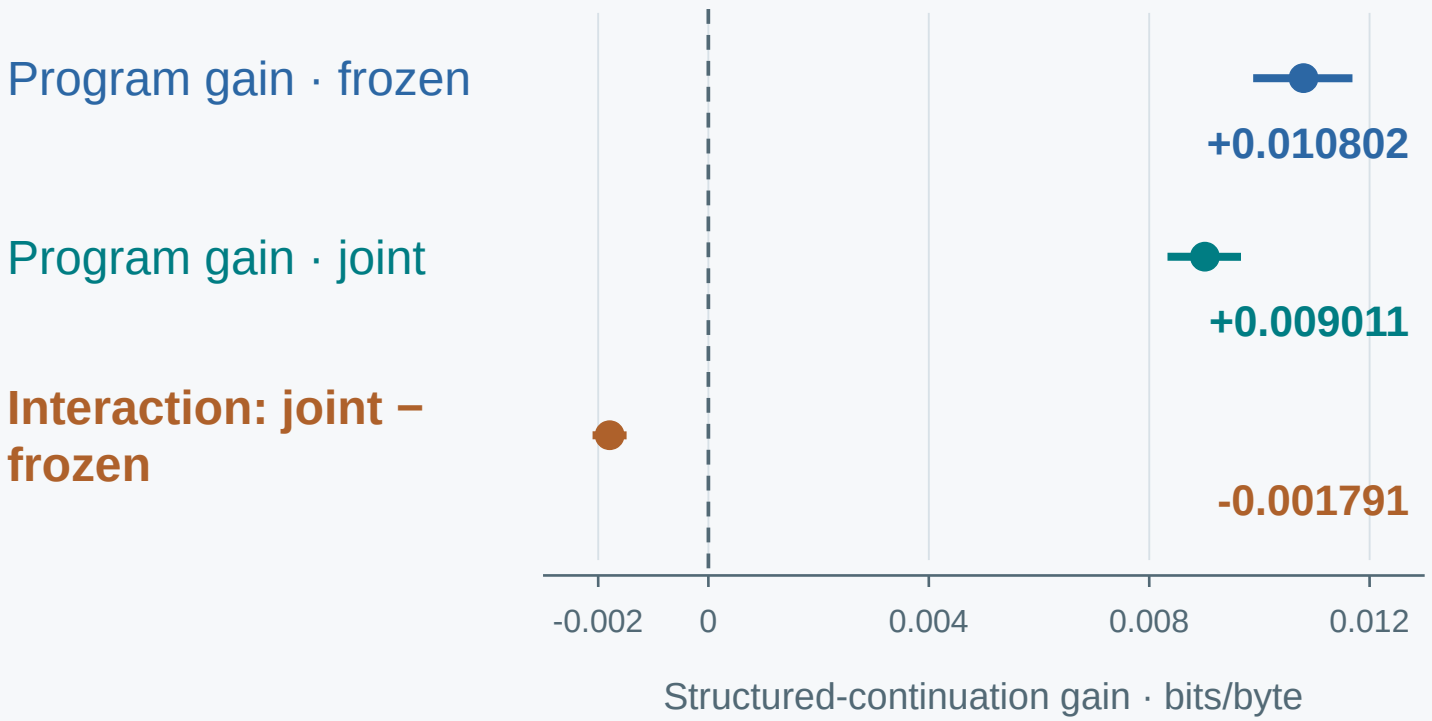


Full programs reduce loss by 8.20% (frozen) and 6.93% (joint) relative to their matched no-program controls.

Source: v44 saved metric CSV; Paper 3 §11. Three-seed means; all 29 structured continuation bytes.

Co-training preserves utility—not stronger synergy

Positive program benefit in both regimes; negative interaction between co-training and the program gain.



Joint training

Ordinary prose improves by 0.1711 bits/byte.

Program-assisted structured prediction stays almost level.

I = Gjoint − Gfrozen

Best mixed score ≠ increased symbolic synergy

Source: v44 saved contrast CSV; Paper 3 §11. Paired 95% intervals condition on the three realized seeds.

Retained runtime use is not feature-free transfer

Intervene after training: keep weights, readout, and correct neural graph observations fixed.



Raw-core transfer

Advantage from prior program training:

−0.000075

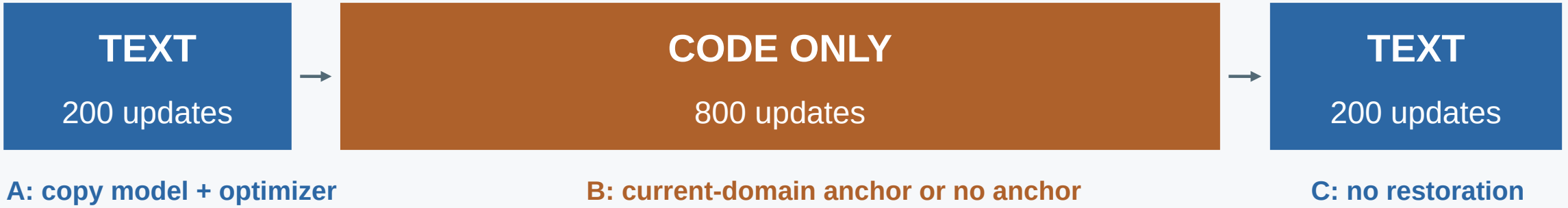
95% interval spans zero. No positive feature-free transfer established.

Swapping preserves the empirical feature marginal while breaking world-specific alignment.

Source: v44 saved contrast CSV; Paper 3 §11. Frozen-model interventions; complete structured continuations.

Text → code → text: a no-replay stress test

Twelve trajectories: three seeds × programs/no programs × anchored/unanchored forks.



Genuine adaptation

199,320 update-eligible parameters; lower trunk remains frozen.

No cross-domain replay

No text examples in B; no code examples in C; optimizer moments continue.

Held-out code

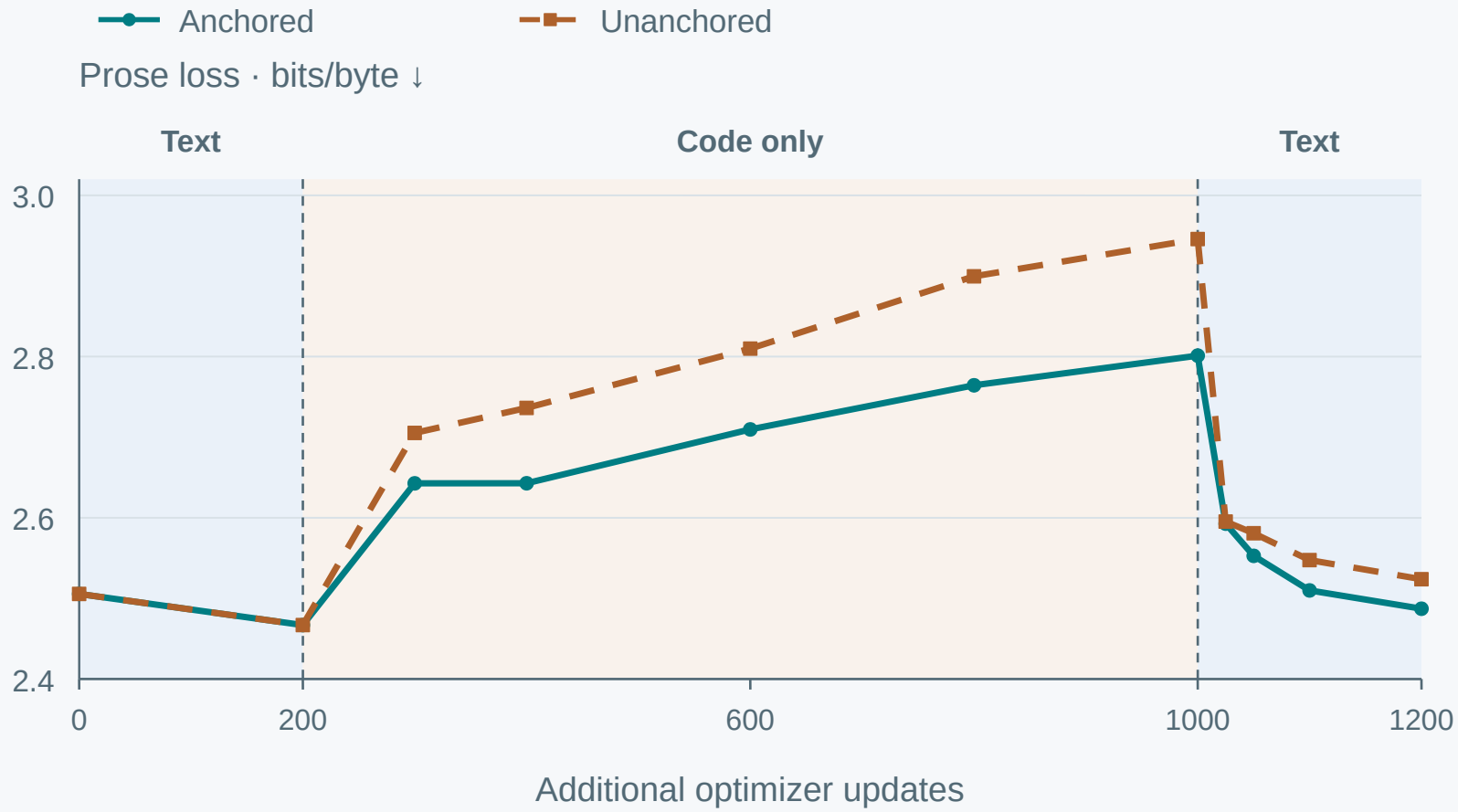
85 train / 20 test modules; Python comments and docstrings removed.

Anchoring uses the pre-code predictor on current-domain inputs. It is not stored-example replay.

Source: v45 protocol; Paper 3 §12. A/B/C denote additional text/code/text adaptation, not pretraining.

Forgetting is real; recovery is substantial

Held-out prose · program-equipped system · all 29 continuation bytes · mean of three seeds



1.45 / 1.51 bpb

Held-out code improvement anchored / unanchored

94% / 88%

Prose deterioration reversed after 200 return updates

42% / 38%

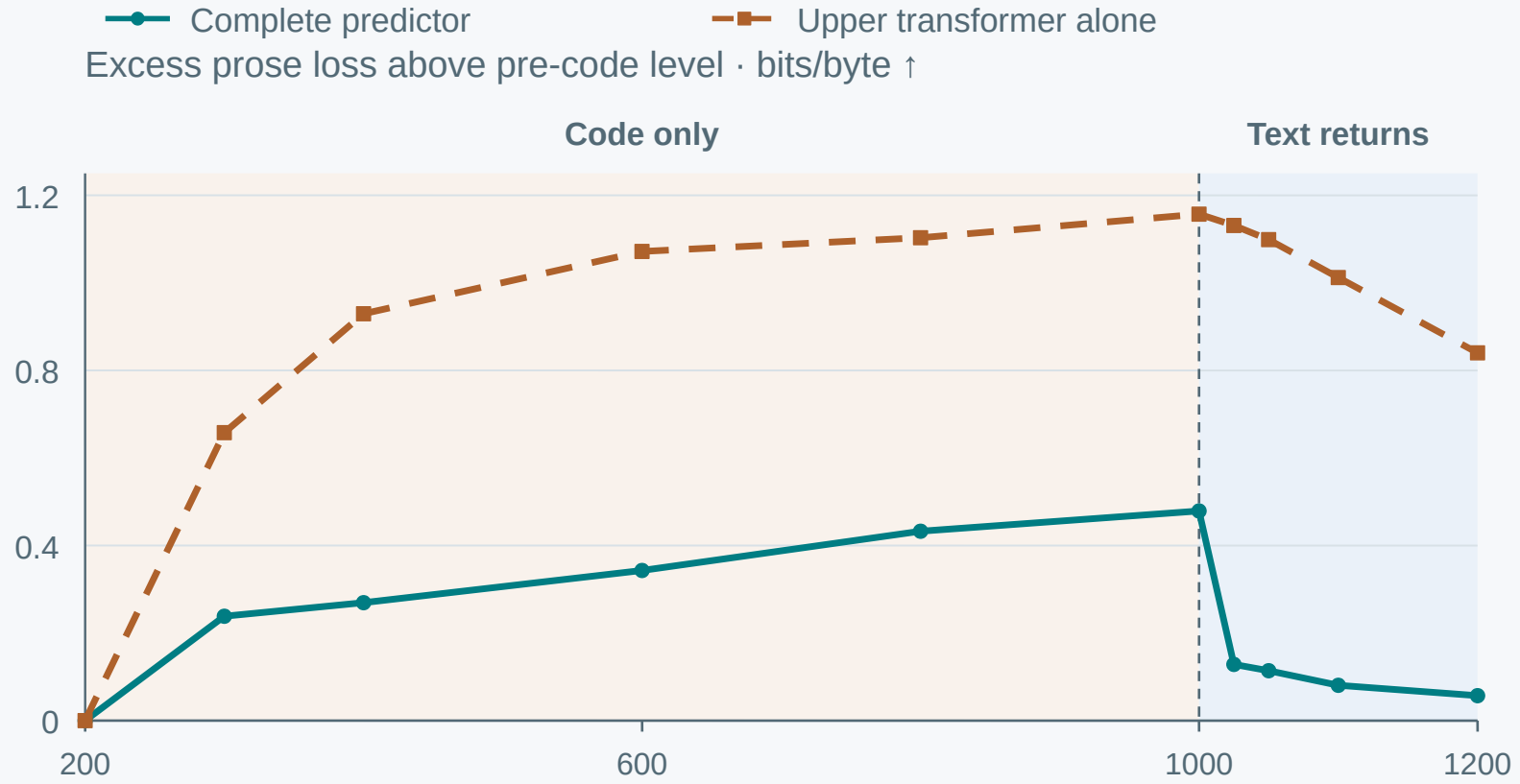
Code-learning gain retained after returning to text

Recovery is not simultaneous retention of both domains.

Source: v45 saved trajectory/endpoint CSVs and ratios JSON; Paper 3 §12. Three-seed means.

Recovered output can hide internal forgetting

Unanchored condition · change in prose loss relative to each predictor's own pre-code endpoint



After text returns

Full predictor: +0.057 bpb
Upper alone: +0.840 bpb

Preserved readers + adaptive routing may compensate.

Their causal shares have not been isolated.

Component diagnostic—not a separately trained matched-transformer baseline.

Source: v45 saved trajectory/endpoint CSVs; Paper 3 §12. Unanchored; each curve subtracts its own A loss.

Useful evidence needs a capable consumer

Promising mechanisms should earn separate claims—not inherit them from the architecture's name.

Preserve predictive evidence

Distributions, support and reliable bindings—not merely feature IDs.

Evaluate conditional utility

Refit the consumer; refresh residuals; keep an equal-budget no-growth control.

Let shared identity do work

A same-witness join is meaningful. A deeper-looking hierarchy may be redundant.

Keep the evidence learning

Update counts and coverage as neural training progresses. A larger bank alone is insufficient.

Separate inference and learning

Test PC credit assignment against BP through the same finite inference.

Measure retained function

Program storage ≠ useful program use. Output recovery ≠ absence of internal forgetting.

Sources: Paper 1 §6; Paper 3 §13; Paper 4 §§2–4. Lessons synthesize different experimental settings.

Scale reliable computation and extraction in parallel

Proposed programme: a stronger common backbone, two task families, and independently replicated controls.

Track R · reliable bindings

Code scope / data flow / event logs
Held-out compositions, distractors and delays
Strong neural controls see the same facts

Track L · extraction uncertainty

Paraphrases, aliases, missing and noisy edges
Hard vs uncertain binding interfaces
Verified-input branch diagnoses the front end

10–30M

Controlled replication
1–2K context · 5 independent runs

→

100–350M

Realistic text / code corpora
2–8K context · cost / memory audits

→

1–3B

Selective integration
Only after smaller-scale gates pass

Primary gate: full-continuation program gain over strong same-fact controls, with latency and memory charged.

Source: Paper 4 §§5–6 and §11. All larger scales and gates are proposals, not achieved results.

Next milestone: a decisive larger experiment

Build a reproducible modular reference, then test two useful task families—not an enormous architecture sweep.

Neural + PC workstream

Upper adaptation and calibration
Finite-inference BP vs PC
Stationarity, gradients and throughput

Programs + search

Typed interpreter and binding store
Conditional birth / refit / admission
Coverage, provenance and cost

Independent evaluation

Same-fact neural controls
Fresh composition and code tests
Repeated-domain retention

Can reusable symbolic computation keep paying as the neural model gets stronger?

Source: Paper 4 §§9–12. Proposed collaboration programme; no new experimental claim.

Exact reference: parameters and interfaces

H3 v43–v45 · one specific instantiation of the general NESS interfaces

Component	Parameters
Frozen deployed trunk	254,400
Adaptive upper student	74,976
Graph memory	33,280
Soft composer	7,216
Phase embedding	232
Reliability network	23,040
Core vocabulary correction	44,320
Program adapter	16,256
Total deployed tensors	**453,720**

Consumer context: 202

96 neural + 64 graph + 32 composer
+ 2 mode + 8 phase

Key network shapes

Reliability: 298–64–48–16
Core correction: 202–96–256
Adapter: 202–32 then 40–32–256
8 feature slots; 16 reader slots

Source: IMPLEMENTATION_AUDIT.md; Paper 2 §5. Teacher is a training reference, not fully deployed.

v44: full endpoints and what “gain” means

All losses are bits per byte; mixed = 80% structured + 20% ordinary prose.

Core / program features	Mixed	Structured	Prose	New names
Frozen / none	0.629512	0.131709	2.620724	0.132852
Frozen / discovered	**0.620871**	**0.120908**	**2.620724**	**0.121588**
Frozen / two-path	0.626859	0.128393	2.620724	0.129553
Frozen / corrupted	0.629510	0.131707	2.620724	0.132828
Joint / none	0.593896	0.129987	2.449531	0.130855
Joint / discovered	**0.586705**	**0.120975**	**2.449622**	**0.121755**
Joint / two-path	0.591554	0.127057	2.449542	0.128118
Joint / corrupted	0.593908	0.130002	2.449533	0.130830

G = L(no programs) − L(discovered). I = Gjoint − Gfrozen = −0.001791 structured bits/byte.

Source: v44 saved metric/contrast CSVs; Paper 3 §11. Rounded to six decimals.

PC: a finite algorithm is not an equilibrium theorem

Earlier nonlinear H3 activity caps differ from today's reliability-only inference.

$$E_0(a,b,z) = \tfrac{1}{2}[\|a - f_1(c)\|^2 + \|b - f_2(a)\|^2 + \|z - f_3(b)\|^2]$$
$$- \lambda \times \text{mean}_i \log(\textstyle\sum_j \text{softmax}(z)_j M_{ij})$$

Training: $E_{\pm\beta} = E_o \pm \beta\ell$; use centered fixed-state parameter partials.

Original H3 audit	Cosine to BP	Relative error
32 settling steps	0.941	0.356
128 settling steps	0.992	0.136

Writable-interface warning

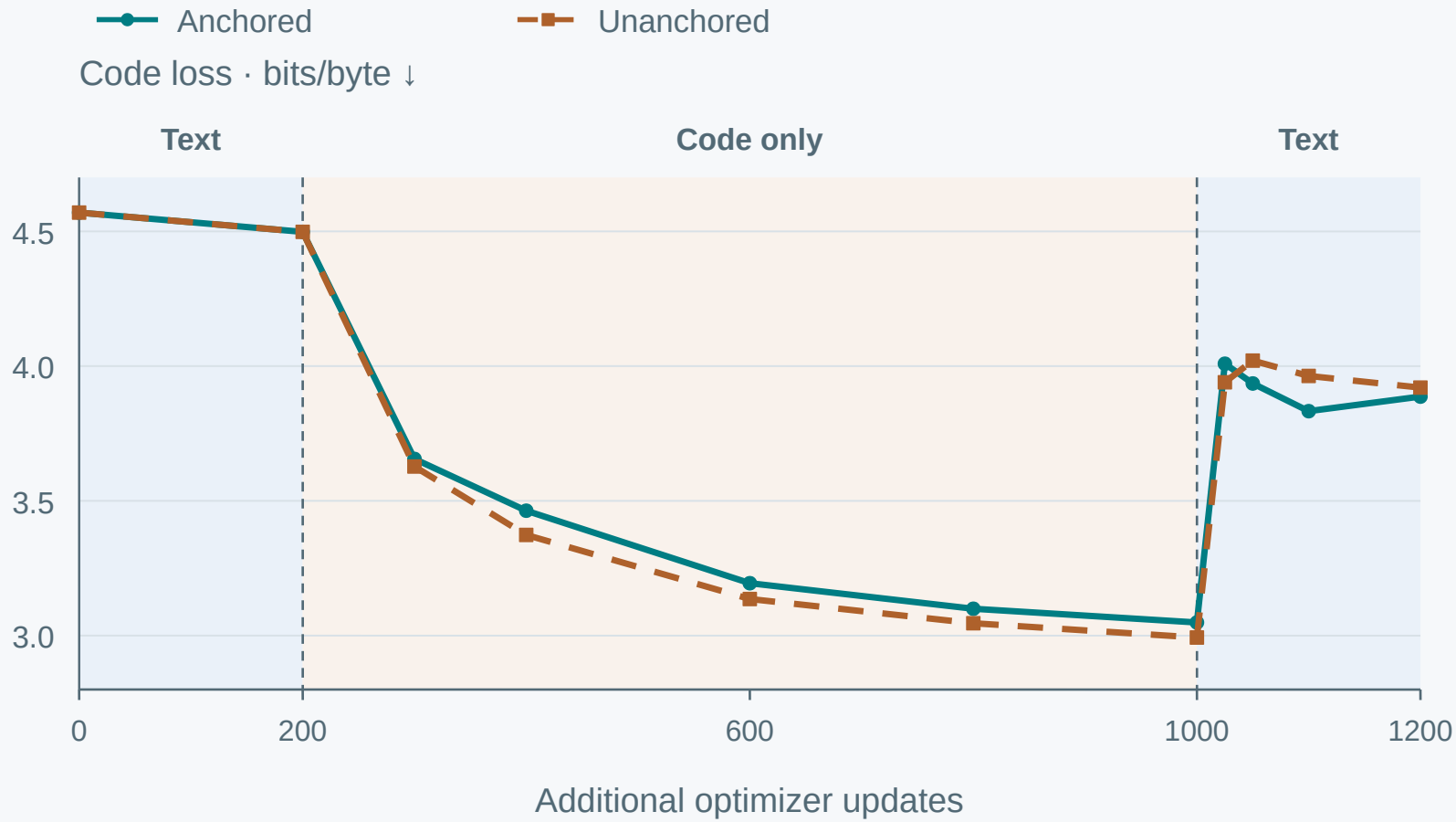
One 32-step audit: cosine 0.267 to equilibrium gradient; no useful predictive gain.

Compare equal finite unrolls, inspect convergence separately, and measure the final predictive objective.

Sources: Paper 2 §8; original H3 and co-distillation numerical audits. Different settings are not pooled.

Code learning—and reverse forgetting

Held-out Python source continuation · same checkpoints and forks as the prose figure



Code gain retained

42.17% with anchoring
38.39% without anchoring

What this is not

29-byte local continuation is not evidence of executable-program correctness.

All 20 held-out modules improve during code training, including on non-whitespace targets.

Source: v45 saved trajectory/endpoint CSVs and ratios JSON; Paper 3 §12. Three-seed means.

Symbolic help survives—but weakens temporarily

Program-feature removal cost on complete binding continuations; higher cost means more retained runtime contribution.



Still useful at B

Positive removal cost in every seed.

Important scope

No new code-specific binding programs were learned.

Source: v45 saved endpoint CSV and contrasts JSON; Paper 3 §12. Fixed-model feature interventions.

A causal test of retention, not just recovery

Repeated narrative → code-with-bindings → event-log cycles, with old domains scored throughout.

Factor	Controlled contrast	What it tests
Preserved evidence	Reader bank / no bank	Does stable predictive memory help?
Consumer plasticity	Adaptive / frozen combiner	Does rerouting explain recovery?
Program learning	Old bank / old + new programs	Does new symbolic learning interfere?
Neural plasticity	Frozen / partially unfrozen trunk	How much protection comes from freezing?
Reference baselines	Same-fact neural model; fixed replay budget	Retention at matched access and cost

Measure forgetting before return training, simultaneous competence, recovery examples, and functional program use.

Source: Paper 4 §9. Proposed causal study; do not run the full Cartesian product immediately.

Execute the join as a set intersection

Exact program-681 semantics, written as readable paths rather than twenty-entity bit operations.

```
trusted = follow(actor, [trusts])
helpful = follow(actor, [inverse_helps])
access  = follow(object,
   [inside, inverse_opens, inverse_holds])

return nonempty(trusted & helpful & access)
```

Before the swap

{Ivo} ∩ {Ivo} ∩ {Kai, Ivo}
= {Ivo} → true

After the swap

{Eli} ∩ {Ivo} ∩ {Kai, Ivo}
= empty → false

All three paths find someone in both worlds. Only the original world has one person common to all three.

Sources: v43 programs.json and selected43.py; replay checks in qa/example_verification.json. Pseudocode is equivalent to bitset execution.

What was learned—and what was supplied

Keep the learned computation, engineered interface, and illustrative case study distinct.

Learned / selected

Lexical observer refinements
Next-byte count distributions
Predictively selected joins
Neural use of program outputs

Supplied by the task

Typed relation vocabulary
Entity identities and records
Mode and event boundary
A bounded path/join grammar

This worked trace

Actual held-out pair and names
Observation-only selection rule
Stored forecasts, three seeds
No new training or model run

The example explains the operation. Aggregate loss, same-fact controls, and interventions establish its measured value.

Sources: original H3 report; v43 archived catalogue, selections, observations and saved forecasts. No new training.

What learns in the current v43–v45 combiner?

Separate the trained neural parameters, the per-prediction inference state, and the frozen symbolic bank.

LEARNED PRIOR

Context + reader diagnostics
298 → 64 → 48 → 16
u = Aφ(context, diagnostics)

→

TEMPORARY INFERENCE

8 past-byte likelihoods
8 reliability-logit updates
w = softmax(z)

→

LEARNED CORRECTION

Context + 8 feature slots
$\delta = \delta_0 + \delta\Pi$
Normalize the final forecast

$$p(y) = \mathrm{softmax}[\ \log(\Sigma_j\, w_j\, q_j(y)) + \delta_0 + \delta\Pi\ ]$$

After observing y: loss = −log p(y) + distillation anchors + small penalties

BP through the finite unroll updates the prior, correction heads, and upper student in joint runs.
Program syntax, lexical tables, lower trunk, and fixed readout stay unchanged in v44–v45.

Sources: Paper 2 §§4–5, §8.1; implementation audit. BP updates parameters; inference updates temporary logits.

Observed outcomes assign soft credit to strategies

For a normalized mixture: $p(y) = \Sigma_s g_s q_s(y)$. Fix the strategy template; learn the gate and evidence coefficients.

$$r_s = g_s\, q_s(y^*) / p(y^*)$$

Responsibility: which strategy explained the observed byte?

$$\partial[-\log p(y^*)] / \partial \text{gate_logit}_s = g_s - r_s$$

Better-than-prior explanation → more predictive credit

ILLUSTRATIVE TWO-STRATEGY CASE

	S1	S2
Prior gate	0.5	0.5
P(observed byte)	0.8	0.2
Responsibility	**0.8**	**0.2**

Evidence logic inside each regime

$$q_s(y) \propto q_0(y) \exp(\Sigma_j \theta_{sj} F_j(y))$$

Signed θ learns promotion, suppression, and overlap correction for explicit factors.

Likelihood training learns weights within fixed logic. Searching new strategy structures is a separate outer operation.

Source: finite-mixture identity; H3 v16 / Paper 2 §4.2. Two-strategy numbers are illustrative, not measured.

Sources, lineage, and interpretation

The presentation is a synthesis of completed records—not a new training run or one pooled benchmark.

P1 **Architecture and Working Principles**

Five interfaces, information flow and core lessons.

P2 **Mathematical Framework and Implemented Instantiation**

Causal contracts, grammar, inference, utility and estimands.

P3 **Experimental Record and Evidence Audit**

Historical studies, v43–v45, model sizes, controls and caveats.

P4 **Lessons, Mechanisms, and a Scaling Programme**

Mechanism interpretation and proposed larger experiments.

Included: editable deck, timed notes, source data, build script, and source hashes. Main talk: 24 slides / 20 minutes.

Sources: four NESS working papers, implementation audit, saved v44/v45 aggregate records.